

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**BLM4522 - AĞ TABANLI PARALEL DAĞITIM
SİSTEMLERİ**

PROJE - 2

Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Şevval Ensarioğlu - 22290742

GitHub Linki:

<https://github.com/SevvalEnsarioglu/blm4522-proje2-db-backup-and-recovery>

İçindekiler

1 Giriş	3
1.1 Projenin Amacı	3
1.2 Kullanılan Araçlar ve Teknolojiler	3
2 Kullanılan Veritabanı: Northwind	4
2.1 Kullanılan Ana Tablolar	4
3 Yedekleme ve Kurtarma Mimarisi	5
3.1 Yedekleme Şemasının Oluşturulması	5
3.2 Yedekleme Geçmişi Tablosu	5
3.3 Recovery Point Tablosu	5
4 Tam Yedekleme Stratejisi	6
4.1 Tam Yedekleme SQL Scripti	6
4.2 Tam Yedekleme Açıklaması	6
5 Fark ve Artık Yedekleme Stratejisi	7
5.1 Change Log Tablosu	7
5.2 Products Tablosu İçin Trigger Function	7
5.3 Trigger Oluşturma	8
5.4 Değişiklik Testi	9
6 Zamanlayıcılarla Yedekleme Planı	10
6.1 Backup Schedule Tablosu	10
6.2 Yedekleme Planlarının Eklenmesi	10
6.3 Yedekleme Planı Özeti	11
7 Felaketten Kurtarma Senaryosu	12
7.1 Felaket Öncesi Recovery Point Oluşturma	12
7.2 Yanlışlıkla Veri Silme	12
7.3 Felaket Senaryosu Açıklaması	13

8 Silinen Verilerin Geri Yüklenmesi	14
8.1 Orders Tablosunu Geri Yükleme	14
8.2 Order Details Tablosunu Geri Yükleme	14
8.3 Restore İşlemini Loglama	14
9 Point-in-Time Restore Mantığı	16
9.1 Güvenli Durumun Kopyalanması	16
9.2 Hatalı Güncelleme Senaryosu	16
9.3 Eski Duruma Geri Döndürme	16
10 Test Yedekleme Senaryoları	18
10.1 Doğrulama Tablosu	18
10.2 Products Yedeğini Test Etme	18
10.3 Orders Yedeğini Test Etme	19
10.4 Doğrulama Sonuçlarını Görüntüleme	19
11 Database Mirroring Simülasyonu	20
11.1 Mirror Şemasının Oluşturulması	20
11.2 Mirror Tablolarının Oluşturulması	20
11.3 Mirror İşlemini Loglama	20
12 Raporlama Sorguları	22
12.1 Yedekleme Geçmişi	22
12.2 Recovery Point Kayıtları	22
12.3 Değişiklik Logları	22
12.4 Zamanlayıcı Kayıtları	22
12.5 Mirror Karşılaştırması	22
13 Sonuç ve Değerlendirme	24
13.1 Yapılan Çalışmaların Özeti	24
13.2 Öğrenilen Dersler	24
13.3 Kaynaklar	25

1 Giriş

Bu proje, BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında hazırlanmıştır. Proje kapsamında PostgreSQL üzerinde çalışan Northwind örnek veritabanı için yedekleme ve felaketten kurtarma planı tasarlanmıştır.

Çalışmada tam yedekleme, fark/artık yedekleme mantığı, zamanlayıcı ile yedekleme planlaması, yanlışlıkla veri silme senaryosu, silinen verilerin geri yüklenmesi, point-in-time restore mantığı ve yedek doğrulama işlemleri SQL komutları ile simüle edilmiştir.

1.1 Projenin Amacı

- Veritabanı yedekleme stratejisinin tasarlanması
- Tam yedekleme mantığının SQL ile uygulanması
- Fark ve artık yedekleme süreçlerinin trigger yapısı ile gösterilmesi
- Felaket senaryosu oluşturarak veri kaybı durumunun simüle edilmesi
- Silinen verilerin yedekten geri yüklenmesi
- Point-in-time restore mantığının örnek bir senaryo ile gösterilmesi
- Yedeklerin doğruluğunun test edilmesi

1.2 Kullanılan Araçlar ve Teknolojiler

Araç	Sürüm	Kullanım Amacı
PostgreSQL	17	Veritabanı yönetim sistemi
DBeaver	—	SQL scriptlerinin çalıştırılması
Northwind DB	—	Örnek e-ticaret veritabanı
Overleaf	—	Proje raporunun hazırlanması

Tablo 1: Kullanılan araçlar ve teknolojiler

2 Kullanılan Veritabanı: Northwind

Northwind veritabanı, bir ticaret şirketinin satış, müşteri, ürün, sipariş ve çalışan bilgilerini modelleyen klasik bir örnek veritabanıdır. Bu projede backup ve recovery senaryolarını uygulamak için Northwind veritabanı kullanılmıştır.

2.1 Kullanılan Ana Tablolar

Tablo	Açıklama
products	Ürün bilgilerinin tutulduğu tablo
orders	Sipariş başlık bilgilerinin tutulduğu tablo
order_details	Sipariş kalemlerinin tutulduğu tablo
customers	Müşteri bilgilerinin tutulduğu tablo
employees	Çalışan bilgilerinin tutulduğu tablo

Tablo 2: Northwind veritabanında kullanılan temel tablolar

3 Yedekleme ve Kurtarma Mimarisi

Projede yedekleme işlemleri için ayrı bir backup_dr şeması oluşturulmuştur. Bu şema içerisinde tam yedek tabloları, değişiklik logları, recovery point kayıtları, yedekleme geçmişi ve yedek doğrulama sonuçları saklanmaktadır.

3.1 Yedekleme Şemasının Oluşturulması

```
CREATE SCHEMA IF NOT EXISTS backup_dr;
```

3.2 Yedekleme Geçmişi Tablosu

```
CREATE TABLE backup_dr.backup_history (  
    backup_id SERIAL PRIMARY KEY,  
    backup_type VARCHAR(30),  
    backup_description TEXT,  
    backup_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) DEFAULT 'SUCCESS'  
);
```

Bu tablo, yapılan tüm yedekleme, geri yükleme ve felaket senaryosu işlemlerinin kayıt altına alınması için kullanılmıştır.

3.3 Recovery Point Tablosu

```
CREATE TABLE backup_dr.recovery_points (  
    point_id SERIAL PRIMARY KEY,  
    point_name VARCHAR(100),  
    description TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Recovery point tablosu, kritik işlemlerden önce sistemin güvenli durumunu temsil eden kurtarma noktalarını kayıt altına almak için kullanılmıştır.

4 Tam Yedekleme Stratejisi

Tam yedekleme, veritabanındaki seçili tabloların tamamının belirli bir anda yedeklenmesi işlemidir. Bu projede products, orders ve order_details tablolarının tam yedekleri alınmıştır.

4.1 Tam Yedekleme SQL Scripti

```
CREATE TABLE backup_dr.products_full_backup AS
SELECT * FROM products;

CREATE TABLE backup_dr.orders_full_backup AS
SELECT * FROM orders;

CREATE TABLE backup_dr.order_details_full_backup AS
SELECT * FROM order_details;

INSERT INTO backup_dr.backup_history (
    backup_type,
    backup_description
)
VALUES (
    'FULL BACKUP',
    'Products, Orders ve Order Details tablolar n n tam
    yede i a l n d .'
);
```

4.2 Tam Yedekleme Açıklaması

Bu işlem sonucunda ana tablolardaki veriler backup_dr şeması altında ayrı yedek tablolara kopyalanmıştır. Böylece herhangi bir veri kaybı durumunda bu tablolardan geri yükleme yapılabilir.

5 Fark ve Artık Yedekleme Stratejisi

Fark ve artık yedekleme, yalnızca değişen kayıtların takip edilmesi mantığına dayanır. Bu projede değişen kayıtlar trigger yapısı ile `incremental_change_log` tablosuna kaydedilmiştir.

5.1 Change Log Tablosu

```
CREATE TABLE backup_dr.incremental_change_log (  
    log_id SERIAL PRIMARY KEY,  
    table_name VARCHAR(100),  
    operation_type VARCHAR(20),  
    record_id TEXT,  
    old_data JSONB,  
    new_data JSONB,  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5.2 Products Tablosu İçin Trigger Function

```
CREATE OR REPLACE FUNCTION backup_dr.log_products_changes()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF TG_OP = 'INSERT' THEN  
        INSERT INTO backup_dr.incremental_change_log (  
            table_name, operation_type, record_id, old_data,  
            new_data  
        )  
        VALUES (  
            'products', TG_OP, NEW.product_id::TEXT, NULL,  
            to_jsonb(NEW)  
        );  
    RETURN NEW;  
END;
```



```

ELSIF TG_OP = 'UPDATE' THEN

    INSERT INTO backup_dr.incremental_change_log (
        table_name, operation_type, record_id, old_data,
        new_data
    )
    VALUES (
        'products', TG_OP, NEW.product_id::TEXT, to_jsonb(
            OLD), to_jsonb(NEW)
        );
    RETURN NEW;

ELSIF TG_OP = 'DELETE' THEN

    INSERT INTO backup_dr.incremental_change_log (
        table_name, operation_type, record_id, old_data,
        new_data
    )
    VALUES (
        'products', TG_OP, OLD.product_id::TEXT, to_jsonb(
            OLD), NULL
        );
    RETURN OLD;

END IF;

RETURN NULL;

END;
$$ LANGUAGE plpgsql;

```

5.3 Trigger Oluşturma

```

CREATE TRIGGER trg_products_incremental_backup
AFTER INSERT OR UPDATE OR DELETE ON products
FOR EACH ROW
EXECUTE FUNCTION backup_dr.log_products_changes();

```

5.4 Değişiklik Testi

```
UPDATE products
SET unit_price = unit_price + 5
WHERE product_id = 1;

SELECT *
FROM backup_dr.incremental_change_log
ORDER BY changed_at DESC;
```

Bu işlem sonucunda `products` tablosundaki değişiklik otomatik olarak log tablosuna kaydedilmiştir. Böylece artık/fark yedekleme mantığı SQL seviyesinde gösterilmiştir.

6 Zamanlayıcılarla Yedekleme Planı

Gerçek sistemlerde yedekleme işlemleri belirli zaman aralıklarında otomatik olarak çalıştırılır. Bu projede zamanlayıcı mantığı SQL tablosu ile simüle edilmiştir.

6.1 Backup Schedule Tablosu

```
CREATE TABLE backup_dr.backup_schedule (  
    schedule_id SERIAL PRIMARY KEY,  
    backup_type VARCHAR(30),  
    frequency VARCHAR(50),  
    scheduled_time TIME,  
    is_active BOOLEAN DEFAULT TRUE,  
    description TEXT  
);
```

6.2 Yedekleme Planlarının Eklenmesi

```
INSERT INTO backup_dr.backup_schedule (  
    backup_type,  
    frequency,  
    scheduled_time,  
    description  
)  
VALUES  
(  
    'FULL BACKUP', 'WEEKLY', '02:00:00', 'Haftalık tam yedekleme planı'),  
(  
    'DIFFERENTIAL BACKUP', 'DAILY', '03:00:00', 'Günlük değişen verilerin yedeklenmesi'),  
(  
    'INCREMENTAL BACKUP', 'HOURLY', '01:00:00', 'Saatlik işlem loglarının yedeklenmesi');
```

6.3 Yedekleme Planı Özeti

Yedek Türü	Sıklık	Saat
Tam Yedekleme	Haftalık	02:00
Fark Yedekleme	Günlük	03:00
Artık Yedekleme	Saatlik	01:00

Tablo 3: Planlanan yedekleme zamanlayıcıları

7 Felaketten Kurtarma Senaryosu

Bu bölümde yanlışlıkla veri silme senaryosu uygulanmıştır. Senaryoda 10248 numaralı sipariş ve bu siparişe ait detay kayıtları silinmiş, ardından tam yedekten geri yüklenmiştir.

7.1 Felaket Öncesi Recovery Point Oluşturma

```
INSERT INTO backup_dr.recovery_points (
    point_name ,
    description
)
VALUES (
    'RP_BEFORE_DELETE_ORDER',
    'Order silme i leminden hemen nce olu turulan recovery
    point.'
);
```

7.2 Yanlışlıkla Veri Silme

```
DELETE FROM order_details
WHERE order_id = 10248;

DELETE FROM orders
WHERE order_id = 10248;

INSERT INTO backup_dr.backup_history (
    backup_type ,
    backup_description
)
VALUES (
    'DISASTER SCENARIO',
    '10248 numaral sipari yanl l kla silindi.'
);
```

7.3 Felaket Senaryosu Açıklaması

Bu işlem, gerçek sistemlerde yaşanabilecek yanlışlıkla veri silme durumunu temsil eder. Sipariş başlığı ve sipariş detayları silindiği için veri bütünlüğü bozulmuştur. Bu nedenle recovery işlemi uygulanmıştır.

8 Silinen Verilerin Geri Yüklenmesi

Silinen veriler, daha önce alınmış olan tam yedek tablolarından geri yüklenmiştir.

8.1 Orders Tablosunu Geri Yükleme

```
INSERT INTO orders
SELECT *
FROM backup_dr.orders_full_backup
WHERE order_id = 10248
AND NOT EXISTS (
    SELECT 1
    FROM orders
    WHERE orders.order_id = 10248
);
```

8.2 Order Details Tablosunu Geri Yükleme

```
INSERT INTO order_details
SELECT *
FROM backup_dr.order_details_full_backup
WHERE order_id = 10248
AND NOT EXISTS (
    SELECT 1
    FROM order_details od
    WHERE od.order_id = backup_dr.order_details_full_backup.
        order_id
    AND od.product_id = backup_dr.order_details_full_backup.
        product_id
);
```

8.3 Restore İşlemini Loglama

```
INSERT INTO backup_dr.backup_history (  
    backup_type ,  
    backup_description  
)  
VALUES (  
    'RESTORE',  
    '10248 numaralı sipariş ve detayları tam yedekten geri  
    yüklenmiştir.'  
);
```

Sonuç: Silinen sipariş ve sipariş detayları başarılı şekilde geri yüklenmiştir.

9 Point-in-Time Restore Mantığı

Point-in-time restore, veritabanının belirli bir zamandaki güvenli durumuna geri döndürülmesi işlemidir. Bu projede bu mantık ürün fiyatı üzerinde yapılan hatalı güncelleme ile simüle edilmiştir.

9.1 Güvenli Durumun Kopyalanması

```
CREATE TABLE backup_dr.products_pitr_demo AS
SELECT * FROM products;
```

9.2 Hatalı Güncelleme Senaryosu

```
UPDATE products
SET unit_price = 999
WHERE product_id = 2;

INSERT INTO backup_dr.backup_history (
    backup_type,
    backup_description
)
VALUES (
    'POINT-IN-TIME TEST',
    'Product 2 zerinde hatal fiyat gncellemesi yap ld .'
);
```

9.3 Eski Duruma Geri Döndürme

```
UPDATE products p
SET
    product_name = b.product_name,
    supplier_id = b.supplier_id,
    category_id = b.category_id,
```

```
quantity_per_unit = b.quantity_per_unit ,
unit_price = b.unit_price ,
units_in_stock = b.units_in_stock ,
units_on_order = b.units_on_order ,
reorder_level = b.reorder_level ,
discontinued = b.discontinued
FROM backup_dr.products_pitr_demo b
WHERE p.product_id = b.product_id
AND p.product_id = 2;
```

Bu işlem ile ürün fiyatı hatalı güncellemeden önceki güvenli değerine döndürülmüştür.

10 Test Yedekleme Senaryoları

Yedekleme işlemlerinin doğruluğunu test etmek için orijinal tablolar ile yedek tabloların kayıt sayıları karşılaştırılmıştır.

10.1 Doğrulama Tablosu

```
CREATE TABLE backup_dr.backup_validation_results (  
    validation_id SERIAL PRIMARY KEY,  
    table_name VARCHAR(100),  
    original_count INT,  
    backup_count INT,  
    validation_status VARCHAR(20),  
    checked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

10.2 Products Yedeğini Test Etme

```
INSERT INTO backup_dr.backup_validation_results (  
    table_name,  
    original_count,  
    backup_count,  
    validation_status  
)  
SELECT  
    'products',  
    (SELECT COUNT(*) FROM products),  
    (SELECT COUNT(*) FROM backup_dr.products_full_backup),  
    CASE  
        WHEN (SELECT COUNT(*) FROM products) =  
            (SELECT COUNT(*) FROM backup_dr.  
                products_full_backup)  
        THEN 'VALID'  
        ELSE 'CHECK REQUIRED'
```

```
END;
```

10.3 Orders Yedeğini Test Etme

```
INSERT INTO backup_dr.backup_validation_results (  
    table_name ,  
    original_count ,  
    backup_count ,  
    validation_status  
)  
SELECT  
    'orders',  
    (SELECT COUNT(*) FROM orders),  
    (SELECT COUNT(*) FROM backup_dr.orders_full_backup),  
    CASE  
        WHEN (SELECT COUNT(*) FROM orders) =  
            (SELECT COUNT(*) FROM backup_dr.orders_full_backup)  
        THEN 'VALID'  
        ELSE 'CHECK REQUIRED'  
    END;  
END;
```

10.4 Doğrulama Sonuçlarını Görüntüleme

```
SELECT *  
FROM backup_dr.backup_validation_results;
```

Sonuç: Yedek doğrulama işlemleri ile alınan yedeklerin veri bütünlüğü kontrol edilmiştir.

11 Database Mirroring Simülasyonu

Database mirroring, ana veritabanının başka bir ortamda eşlenmiş bir kopyasının tutulmasıdır. Bu projede mirror_db şeması oluşturularak temel tabloların ayna kopyaları alınmıştır.

11.1 Mirror Şemasının Oluşturulması

```
CREATE SCHEMA mirror_db;
```

11.2 Mirror Tablolarının Oluşturulması

```
CREATE TABLE mirror_db.products_mirror AS
SELECT * FROM products;

CREATE TABLE mirror_db.orders_mirror AS
SELECT * FROM orders;

CREATE TABLE mirror_db.order_details_mirror AS
SELECT * FROM order_details;
```

11.3 Mirror İşlemini Loglama

```
INSERT INTO backup_dr.backup_history (
    backup_type ,
    backup_description
)
VALUES (
    'DATABASE MIRRORING',
    'Products, Orders ve Order Details tablolar mirror_db
    emasna kopyaland .'
);
```

Bu yapı sayesinde ana veritabanındaki kritik tabloların farklı bir şema altında yedek kopyaları tutulmuştur.

12 Raporlama Sorguları

Proje sonunda yedekleme geçmişi, recovery point kayıtları, değişiklik logları, zamanlayıcı planı ve doğrulama sonuçları raporlanmıştır.

12.1 Yedekleme Geçmişi

```
SELECT *  
FROM backup_dr.backup_history  
ORDER BY backup_time;
```

12.2 Recovery Point Kayıtları

```
SELECT *  
FROM backup_dr.recovery_points  
ORDER BY created_at;
```

12.3 Değişiklik Logları

```
SELECT *  
FROM backup_dr.incremental_change_log  
ORDER BY changed_at;
```

12.4 Zamanlayıcı Kayıtları

```
SELECT *  
FROM backup_dr.backup_schedule;
```

12.5 Mirror Karşılaştırması

```
SELECT  
    p.product_id,
```

```
    p.product_name ,  
    p.unit_price AS original_price ,  
    m.unit_price AS mirror_price  
FROM products p  
JOIN mirror_db.products_mirror m  
    ON p.product_id = m.product_id  
ORDER BY p.product_id  
LIMIT 10;
```


13 Sonuç ve Değerlendirme

Bu proje kapsamında PostgreSQL üzerinde Northwind veritabanı için SQL tabanlı yedekleme ve felaketten kurtarma planı hazırlanmıştır. Proje boyunca tam yedekleme, fark/artık yedekleme, zamanlayıcı mantığı, felaket senaryosu, geri yükleme, point-in-time restore ve database mirroring işlemleri uygulanmıştır.

13.1 Yapılan Çalışmaların Özeti

Başlık	Yapılan İşlem
Tam Yedekleme	Kritik tablolar backup_dr şemasına kopyalandı
Fark/Artık Yedekleme	Trigger ile değişiklik loglama yapısı kuruldu
Zamanlayıcı	Backup schedule tablosu ile planlama simüle edildi
Felaket Senaryosu	10248 numaralı sipariş yanlışlıkla silindi
Geri Yükleme	Silinen sipariş tam yedekten geri getirildi
Point-in-Time Restore	Hatalı fiyat güncellemesi eski haline döndürüldü
Database Mirroring	Kritik tablolar mirror_db şemasına kopyalandı
Test Senaryoları	Yedek ve ana tablo kayıt sayıları karşılaştırıldı

Tablo 4: Proje kapsamında yapılan işlemler

13.2 Öğrenilen Dersler

- Veritabanı yedekleme planı, veri güvenliği için kritik öneme sahiptir.
- Tam yedekleme, felaket durumlarında temel geri dönüş noktası sağlar.
- Fark ve artık yedekleme, yalnızca değişen verileri takip ederek daha verimli bir yapı sunar.
- Trigger mekanizması, değişikliklerin otomatik olarak loglanmasını sağlar.
- Recovery point kullanımı, kritik işlemlerden önce güvenli geri dönüş noktası oluşturur.

- Yedeklerin alınması kadar doğrulanması da önemlidir.
- Database mirroring, yüksek erişilebilirlik ve veri güvenliği açısından önemli bir yaklaşımdır.

13.3 Kaynaklar

- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- Northwind PostgreSQL: https://github.com/pthom/northwind_psql
- PostgreSQL Tutorial: <https://www.postgresqltutorial.com/>